

Final Project Documentation

Date: 04/14/2026

Group 4: Erin Wright-Vazquez, Jacob Ko, Sadia Rahman

Project Proposal

This project explores how environmental factors influence pitching performance beyond traditional skill based metrics. The goal is to combine performance data with contextual variables such as stadium location, altitude, and weather conditions. By structuring the data in a relational format, the analysis aims to determine whether factors like temperature, wind, and elevation contribute to variability in pitching outcomes.

Data Overview

Dataset	Link	Description
Pybaseball statcast API	pybaseball GitHub Repo: https://github.com/schorm/pybaseball pybaseball 2.0.0 installation guide in python https://pypi.org/project/pybaseball/2.0.0/	Pybaseball is the dynamic primary performance dataset, providing pitch and game level data (e.g, velocity, spin rate, and outcomes) that can be aligned with stadium and weather data for analysis.
Seamheads Park	Seamhead Park Dataset: https://www.seamheads.com/ballparks/	The Seamheads Ballparks database provides static structured stadium data, including location and altitude, which enables linking performance data to environmental context and integrating weather data.
Open-Meteo API	OpenMeteo API: https://open-meteo.com/ openmeteo api installation and overview: https://pypi.org/project/openmeteo-requests/#description	The Open-Meteo API provides dynamic weather data (e.g, temperature, humidity, wind, precipitation) using stadium coordinates and game dates, enabling analysis of how environmental conditions impact pitching performance.
MLB Ballpark (Kaggle)	Kaggle dataset: https://www.kaggle.com/dataset/s/paulrjohnson/mlb-ballparks?resource=download_	The Kaggle MLB Ballparks dataset provides static team associations for each stadium, complementing Seamheads data by linking parks to team identity and enabling a

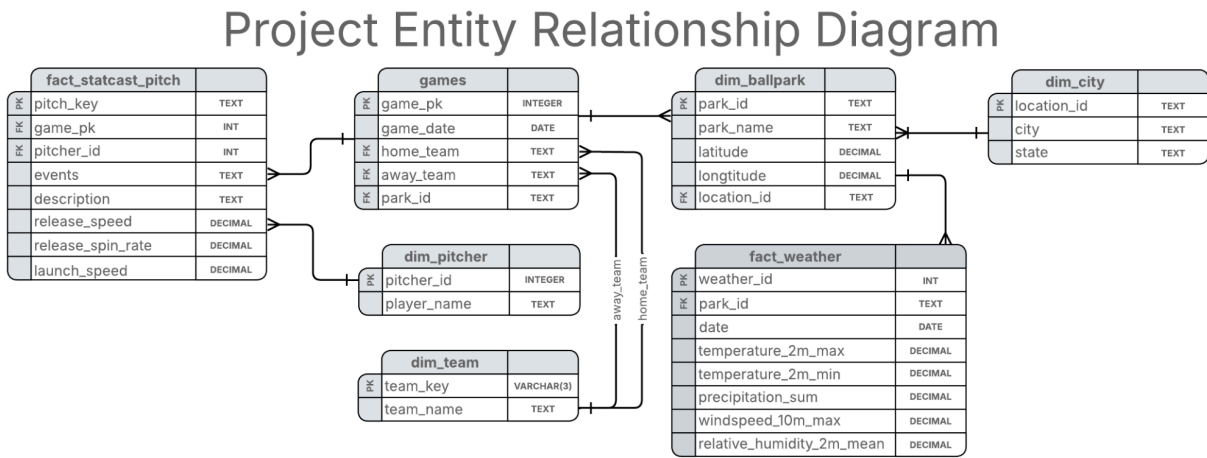
		unified ballpark table.
--	--	-------------------------

Cleaning and Transformation

Data cleaning and transformation were performed across both static datasets (Seamheads and MLB/Kaggle) and ingested dynamic datasets to ensure consistency prior to integration. This process included standardizing column names, normalizing park and team naming conventions, and creating shared keys to resolve discrepancies across sources. Team abbreviations were aligned, and relationships such as team to ballpark mappings and park date combinations were established to support accurate joins. Duplicate and inconsistent records were removed, and key identifiers (including pitch, game, and weather records) were validated for uniqueness.

Following cleaning, the data was structured into a relational format consisting of fact and dimension tables. Pitch level, game level, and weather data were organized into fact tables, while descriptive attributes for pitchers, teams, ballparks, and locations were maintained in dimension tables. Additional transformations, such as deriving pitcher team from inning context and enriching game data with weather information, ensured completeness and consistency across datasets. The final transformed tables were validated for referential integrity and prepared for downstream analysis.

Final ERD



The final ERD organizes the data into separate tables based on key entities such as games, pitchers, teams, ballparks, cities, and weather. A normalized structure was used to reduce redundancy by storing each type of information only once and linking tables through primary and foreign keys. For example, ballpark and city details are stored in dimension tables, while pitch level and weather data are stored in fact tables. The games table serves as a central link connecting teams, ballparks, and performance data.

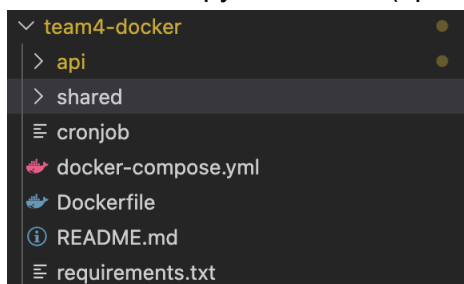
Database and Schema

The database and schema were implemented in SQL by creating separate tables for each entity defined in the ERD, including games, pitchers, teams, ballparks, cities, pitch level data, and weather. Each table was defined with primary keys, and relationships were enforced through foreign keys to connect related data (such as linking pitch data to games and pitchers, and games to teams and ballparks). Dimension tables store descriptive attributes like team and ballpark information, while fact tables store measurable data such as pitch performance and weather observations. Data types were standardized across tables to ensure accurate joins, resulting in a normalized schema that minimizes redundancy and supports efficient querying.

Docker/Running Full Pipeline

Our docker-compose.yml file starts three services: a Postgres container to host our SQL database, an Airflow service for our database automation, and an API service that hosts our API application. All of these work together to produce our end-to-end application. The steps to start our services are below.

1. Unzip files to a directory of your choice and navigate to the `team4-docker` folder. This should contain a docker-compose.yml file along with our other code.
 - a. Make sure that all your files have the correct file extensions. Occasionally, some files may get appended with `.txt`, which may cause errors. The files below should be checked for this.
 - i. Dockerfiles (in both team4-docker and api)
 - ii. cronjob
 - iii. .python-version (api folder)



2. Once inside the above folder, run the following command to build and start the three services. Make sure that you have access to the docker daemon before running this (Docker Desktop must be running).
 - a. `docker compose up --build`
 - b. If you run into issues, the `README.md` contains some helpful debugging tips and how to proceed.
3. To verify that the containers are built correctly, navigate to Docker Desktop. You should see a new container with three running services.

☐	☒	team4-docker	-	-	-	0.56%	12 seconds ago			
☐	☒	postgres-1	4b36114b88af	postgres:11	5432:5432	0.05%	13 seconds ago			
☐	☒	airflow-jupyter	aa3e879a08a4	team4-docker	8080:8080	0%	12 seconds ago			
☐	☒	fastapi-1	2093822fdebb	team4-docker	8000:8000	0.51%	12 seconds ago			

Once these are running, you are ready to interact with the rest of the pipeline.

4. Our second command will call some endpoints from our API. Make sure that the Airflow DAG completes before calling this. To check the status of the DAG, you can navigate to the Airflow web server following the steps in the section below. Note that this may take 13-17 minutes due to the amount of data we ingest.
 - a. ``docker compose exec airflow-jupyter python3 run_app.py``.

```
[{"player_id": 642239, 'pitcher_name': 'Rob Zastryzny'}, {'player_id': 670032, 'pitcher_name': 'Nicky Lopez'}, {'player_id': 596001, 'pitcher_name': 'Jakob Junis'}, {'player_id': 593576, 'pitcher_name': 'Héctor Neris'}, {'player_id': 657248, 'pitcher_name': 'Glenn Otto'}, {'player_id': 608337, 'pitcher_name': 'Lucas Giolito'}, {'player_id': 665871, 'pitcher_name': 'Javier Assad'}, {'player_id': 680694, 'pitcher_name': 'Kyle Bradish'}, {'player_id': 643507, 'pitcher_name': 'Trevor May'}, {'player_id': 663362, 'pitcher_name': 'Matt Waldron'}, {'player_id': 593958, 'pitcher_name': 'Eduardo Rodríguez'}, {'player_id': 621051, 'pitcher_name': 'Steven Wilson'}, {'player_id': 592351, 'pitcher_name': 'Jon Gray'}, {'player_id': 695280, 'pitcher_name': 'Clay Holmes'}, {'player_id': 682010, 'pitcher_name': 'Lane Ramsey'}, {'player_id': 607755, 'pitcher_name': 'Andrew Vasquez'}, {'player_id': 456501, 'pitcher_name': 'Johnny Cueto'}, {'player_id': 607732, 'pitcher_name': 'Jacob Stallings'}, {'player_id': 656271, 'pitcher_name': 'Brock Burke'}, {'player_id': 680723, 'pitcher_name': 'Drew Rom'}, {'player_id': 642545, 'pitcher_name': 'Jaime Barria'}, {'player_id': 666619, 'pitcher_name': 'Gregory Santos'}, {'player_id': 650496, 'pitcher_name': 'J. C. Mejía'}, {'player_id': 680735, 'pitcher_name': 'Austin Cox'}, {'player_id': 669467, 'pitcher_name': 'André Pallante'}
```

To explore our API documentation further, see below.

API Implementation and Automation

Our Airflow DAG is configured to execute the pipeline on a daily schedule. It runs a more comprehensive version of the pipeline that ingests a larger volume of data from Openmeteo and pybaseball consolidated for June 2023 - August 2023, with an average runtime of approximately 13–17 minutes. The DAG definition is defined in the ``dag_run_pipeline.py`` file.

The version run above is a streamlined implementation that executes faster over a smaller dataset. We chose to do this to demonstrate our pipeline capability in a more efficient manner while still showing its effectiveness. In practice, the full pipeline would run during off-peak hours to refresh the database with the most up-to-date data, enabling users to query it later through the API and thereby minimizing the impact of processing latency.

The Airflow DAG should start on its own, but if not, follow the steps below. Keep in mind, since this involves ingesting a lot more data (specifically from the Openmeteo source), this may take awhile to complete.

1. Ensure that the above steps were followed correctly and the airflow-jupyter service is running (i.e., check this is running in Docker Desktop).
2. Open up the Airflow GUI (<http://localhost:8080/>) and log in with the following credentials
 - a. Username: admin
 - b. Password: admin
3. Search for `dag\_run\_pipeline`.
4. Make sure the DAG is running by unpausing it.

To access the data in our SQL database, we implemented a FastAPI application with several endpoints to explore how environmental factors may affect pitching performance. [FastAPI](#) is a modern web framework for Python APIs. One reason we chose this framework over something like Django/Flask is that FastAPI was built to be fast and high-performance. It is also compatible

with other open standards for APIs, such as OpenAPI, which allows documentation to be auto-generated via Swagger docs.

The code for our API is contained inside `/api/main.py`. This code is documented with what each endpoint does and its intended use. Alternatively, the OpenAPI interactive documentation can also be explored following the steps below:

1. Assuming the `team4-docker` container is running, navigate to the following URL: <http://localhost:8000/docs/>. This should redirect you to the Swagger UI interactive docs.



2. To interact with an endpoint, click on it. A brief description is provided for each endpoint, explaining its intent.



3. To query the endpoint, you can click the `'Try it out'` button. This will allow you to enter any parameters, then call the API. The docs then show you how you can call the API through a curl command or via a request URL. It also shows you what the response of the endpoint is.

```

Curl
curl -X 'GET' \
  'http://localhost:8000/pitchers' \
  -H 'accept: application/json'

Request URL
http://localhost:8000/pitchers

Server response
Code    Details
200
Response body
[
  [
    570666,
    "Luis Cessa"
  ],
  [
    642239,
    "Rob Zastryzny"
  ],
  [
    670032,
    "Nicky Lopez"
  ],
  [
    596001,
    "Jakob Junis"
  ],
  [
    593576,
    "Héctor Neris"
  ],
  [
    657248,
    "Glenn Otto"
  ]
]

```

4. To further interact with the API outside of the interactive documentation, you can call the API via `curl` or request URLs.

a. Curl

```

(base) jacob@MacBook-Air ~ % curl -X 'GET' \
  'http://localhost:8000/pitchers' \
  -H 'accept: application/json'
[[{"id": 570666, "name": "Luis Cessa"}, {"id": 642239, "name": "Rob Zastryzny"}, {"id": 670032, "name": "Nicky Lopez"}, {"id": 596001, "name": "Jakob Junis"}, {"id": 593576, "name": "H\u00e9ctor Neris"}, {"id": 657248, "name": "Glenn Otto"}]]

```

b. Request URL

```

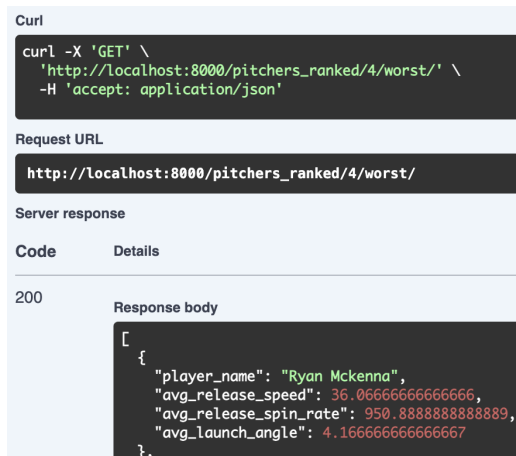
localhost:8000/pitchers
Pretty-print ☒
[
  [570666, "Luis Cessa"],
  [642239, "Rob Zastryzny"],
  [670032, "Nicky Lopez"],
  [596001, "Jakob Junis"],
  [593576, "H\u00e9ctor Neris"]
]

```

Example Workflow

While there are multiple ways one could use our API to answer the project proposal, below details an example workflow of how a user could utilize our API to explore the idea of how weather may affect pitching performance.

1. Use the `pitchers_ranked` endpoint to get a list of the best and worst pitchers by average release speed. This statistic can be used to provide a simple metric to compare “good” vs. “bad” performance.



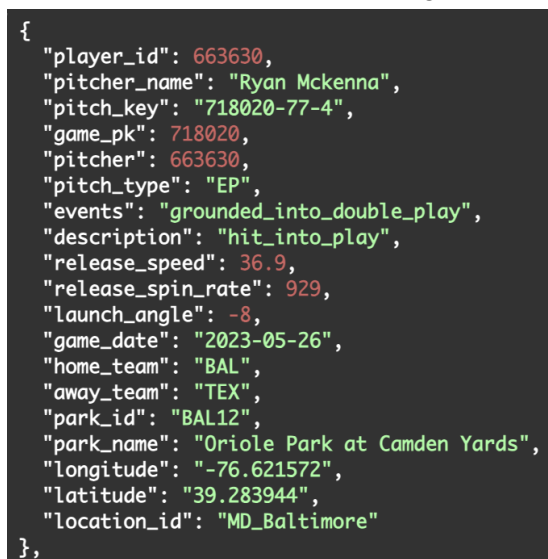
The screenshot shows a web browser interface with a dark theme. At the top, there's a 'Curl' section with a code block containing a curl command: `curl -X 'GET' \ 'http://localhost:8000/pitchers_ranked/4/worst/' \ -H 'accept: application/json'`. Below this is a 'Request URL' section showing `http://localhost:8000/pitchers_ranked/4/worst/`. The 'Server response' section shows a status code of '200' and a 'Response body' containing a JSON array with one object: `{ "player_name": "Ryan Mckenna", "avg_release_speed": 36.066666666666666, "avg_release_spin_rate": 950.8888888888889, "avg_launch_angle": 4.166666666666667 }`.

```
Curl
curl -X 'GET' \
'http://localhost:8000/pitchers_ranked/4/worst/' \
-H 'accept: application/json'

Request URL
http://localhost:8000/pitchers_ranked/4/worst/

Server response
Code    Details
200
Response body
[
  {
    "player_name": "Ryan Mckenna",
    "avg_release_speed": 36.066666666666666,
    "avg_release_spin_rate": 950.8888888888889,
    "avg_launch_angle": 4.166666666666667
  },
]
```

2. Once you decide on two pitchers to compare, use the `pitcher_stats` endpoint to get pitch-level statistics for the pitcher. This includes helpful data points like the outcomes of specific pitches and the specific game in which they occurred.



The screenshot shows a JSON object with various fields: `"player_id": 663630, "pitcher_name": "Ryan Mckenna", "pitch_key": "718020-77-4", "game_pk": 718020, "pitcher": 663630, "pitch_type": "EP", "events": "grounded_into_double_play", "description": "hit_into_play", "release_speed": 36.9, "release_spin_rate": 929, "launch_angle": -8, "game_date": "2023-05-26", "home_team": "BAL", "away_team": "TEX", "park_id": "BAL12", "park_name": "Oriole Park at Camden Yards", "longitude": "-76.621572", "latitude": "39.283944", "location_id": "MD_Baltimore"`.

```
{
  "player_id": 663630,
  "pitcher_name": "Ryan Mckenna",
  "pitch_key": "718020-77-4",
  "game_pk": 718020,
  "pitcher": 663630,
  "pitch_type": "EP",
  "events": "grounded_into_double_play",
  "description": "hit_into_play",
  "release_speed": 36.9,
  "release_spin_rate": 929,
  "launch_angle": -8,
  "game_date": "2023-05-26",
  "home_team": "BAL",
  "away_team": "TEX",
  "park_id": "BAL12",
  "park_name": "Oriole Park at Camden Yards",
  "longitude": "-76.621572",
  "latitude": "39.283944",
  "location_id": "MD_Baltimore"
},
```

3. Then we can use the park name and the game date to determine the weather conditions for that game using the `weather/<park_name>/<date>` endpoint, which retrieves weather data for the park on the specified date.

```
{  
  "park_name": "Oriole Park at Camden Yards",  
  "city": "Baltimore",  
  "state": "MD",  
  "temperature_2m_max": 71.3,  
  "temperature_2m_min": 50.8,  
  "precipitation_sum": 0,  
  "windspeed_10m_max": 10.2,  
  "relative_humidity_2m_mean": 62  
}
```

4. This can be repeated for multiple pitchers to determine if there is a statistical difference in pitching performance at ballparks where the weather conditions are more severe. Of course, this is a simple example, but there are additional endpoints beyond these that could assist in further analysis, such as one that returns the average weather conditions for each ballpark in the database.